Graph Algorithms

```
WHATEVERFIRSTSEARCH(s):
    put s into the bag
    while the bag is not empty
        take v from the bag
        if v is unmarked
            mark v
            for each edge vw
                put w into the bag
```

Depth-first-search:
stack - last in first out

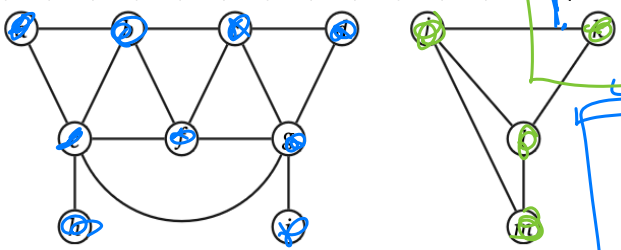
Breadth-first-search:
queue - first in first out

Best-first-search:
priority queue

- Reachability - WFS
- Shortest path (# edges) - BFS
- Shortest path (weight) - Dij-)

breadth ↓

G



```
WFSALL(G):
  for all vertices v
      unmark v
  for all vertices v
      if v is unmarked
          WHATEVERFIRSTSEARCH(v)
```

```
COUNTCOMPONENTS(G):
  count ← 0
  for all vertices v
      unmark v
  for all vertices v
      if v is unmarked
          count ← count + 1
          WHATEVERFIRSTSEARCH(v)
  return count
```

WFS(a)

CountComponents(G)

=> 2

```
COUNTANDLABEL(G):
  count ← 0
  for all vertices v
      unmark v
  for all vertices v
      if v is unmarked
          count ← count + 1
          LABELONE(v, count)
  return count
```

```
⟨Label one component⟩
LABELONE(v, count):
  while the bag is not empty
      take v from the bag
      if v is unmarked
          mark v
          comp(v) ← count
          for each edge vw
              put w into the bag
```